Mo's Algorithm

A difficult problem

Given $A[0, n-1]$

Goal: Answer q queries (OFFLINE)

$$\text{query}(\ell, r) = \sum_{e \in D(\ell, r)} e \cdot K_e^2$$

$D(\ell, r)$ = set of distinct elements in $A[\ell..r]$

K_e = number of occurences of e in $A[\ell..r]$

$$A \quad 2 \quad \underbrace{1 \quad 3 \quad 2 \quad 2 \quad 2 \quad 1} \quad 4$$

$$1 \cdot 2^2 + 3 \cdot 1^2 + 2 \cdot 3^2$$

Target time complexity: $\Theta((n+q)\sqrt{n})$ time

Problem

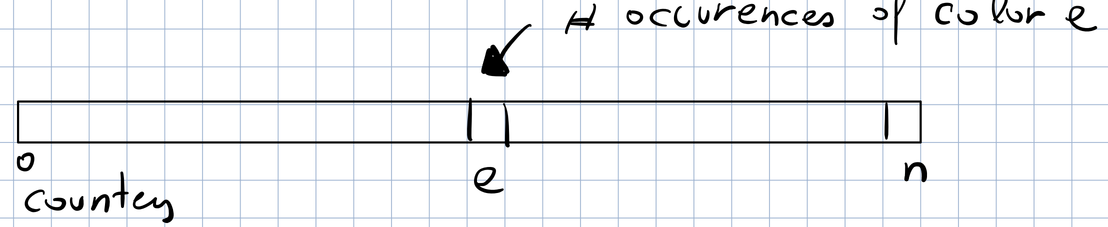
Given $A[1,n]$ of colors (values $\leq n$)

and a set Q of q queries (OFFLINE)

3 or More (i,j): reports # of distinct colors that occur at least 3 times in $A[i, \dots, j]$

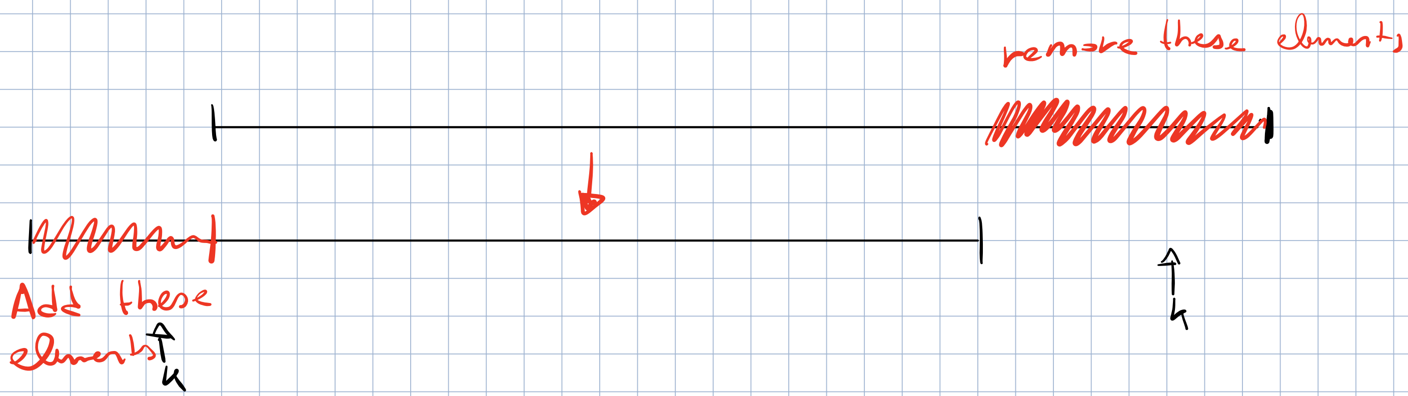
Target time complexity $= \Theta((n+q)\sqrt{n})$ time

Trivial solution $\Theta(n \cdot q)$ time



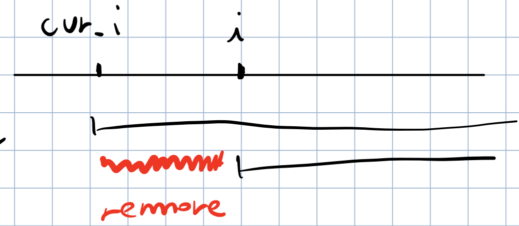

```
for (i,j) in Q:
    answer = 0
    counters = [0] * n
    for k in range(i,j):          ( for e in A[i:j]: )
        counters[A[k]] += 1
        if counters[A[k]] == 3:
            answer += 1
    print(answer)
```

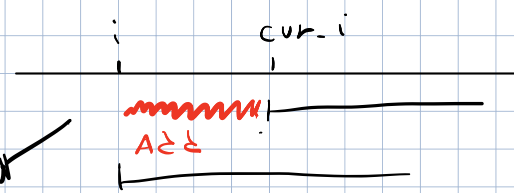
$\Theta(n \cdot q)$ time

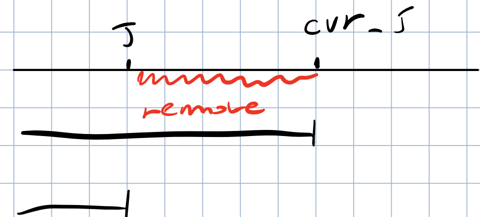


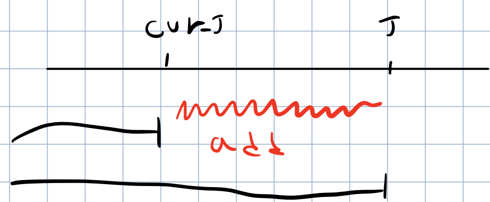

```
def Remove(k):
    counters[A[k]] -= 1
    if counters[A[k]] == 2:
        answer -= 1

def Add(k):
    counters[A[k]] += 1
    if counters[A[k]] == 3:
        answer += 1
```

```
def Solve (A, Q)

    cur_i = 0
    cur_j = 0
    answer = 0
    counters = [0] * n

    for (i, j) in Q:
        while cur_i < i:
            Remove(cur_i)
            cur_i += 1
        while i < cur_i:
            Add(cur_i)
            cur_i -= 1

        while cur_j > j:
            Remove(cur_j)
            cur_j -= 1
        while cur_j < j:
            Add(cur_j)
            cur_j += 1
        print(answer)
```





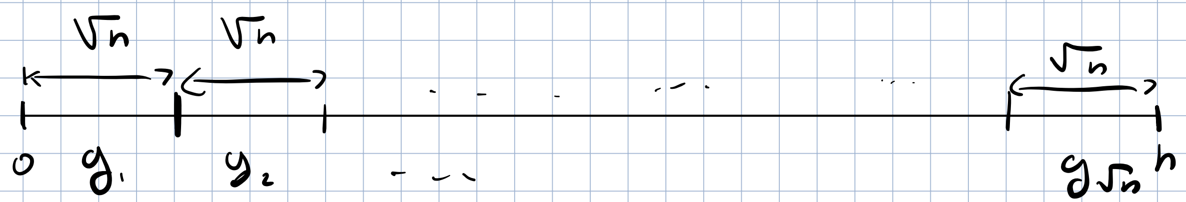




Time Complexity : $\Theta(n \cdot q)$ time

There exists an ordering of the queries such that the time complexity is

$$\Theta\left((n+q)\sqrt{n} \right) \text{ time}$$



- group queries by starting position
- in each group, sort by ending position

Solve queries in $g_1, g_2, g_3, \ldots$

Sort query (i, j) by $(i/\sqrt{n}, j)$

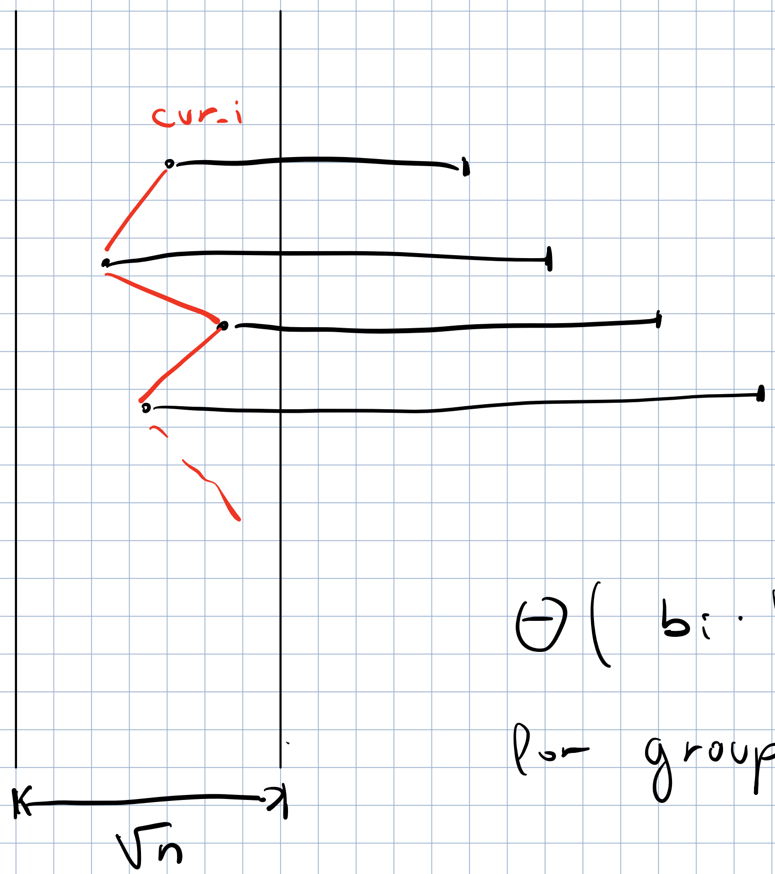
Focus on a group,

First consider cur_j

- we move it by $\leq n$ steps for each group

Overall, $\Theta(n\sqrt{n})$ moves (or calls to Add)

What about moves to cur_i?



$\Theta(b_i \cdot \sqrt{n})$ moves

for group i

$$\Theta\left(\sum_{i=1}^{\sqrt{n}} b_i \cdot \sqrt{n}\right) = \Theta(\sqrt{n}\, q) \text{ moves}$$

$$\Theta(n\sqrt{n} + q\sqrt{n}) = \Theta((n+q)\sqrt{n}) \text{ moves}$$

A difficult problem

Given $A[0, n-1]$

Goal: Answer q queries (OFFLINE)

$$query(\ell, r) = \sum_{e \in D(\ell, r)} e \cdot k_e^2$$

$D(\ell, r)$ = set of distinct elements in $A[\ell \ldots r]$

k_e = number of occurences of e in $A[\ell \ldots r]$

```
def Add(k):
    ke = counter[A[k]]
    counter[A[k]] += 1
    answer -= A[k] · ke^2
    answer += A[k] · (ke+1)^2

def Remove(k):
    ke = counter[A[k]]
    counter[A[k]] -= 1
    answer -= A[k] · ke^2
    answer += A[k] · (ke-1)^2
```